

Loyola Marymount University

CMSI 3300 Artificial Intelligence

Fall 2024

Professor Lanyu Shang

Announcements

- Homework 4
  - Due **this Friday** 12/6 @5 pm
- Extra Credit Opportunities
  - Opportunity 1: Final Question Contribution (on Brightspace)
    - Google Form: https://forms.gle/6cyMNL9w1UdoPEF19
    - Due this Friday 12/6 @5 pm
    - 3 extra points on the exam if you contribute
    - 2 additional points if your question is selected

Announcements

- More Extra Credit Opportunities
  - Opportunity 2: Homework 5 (on Brightspace)
    - 5% of the total grade
  - Opportunity 3: Final Exam Practice (coming soon)
    - 3% of the total grade
  - Grading
    - Individual-based
    - Due **Friday, 12/13 @5 pm**

Final Exam Logistics

- Time: **Thursday 12/12, 2:00PM to 3:30PM**
- Location: **SEA 205**
- **Conceptual and mathematical**
- **Closed** book/notes/computer/Internet/classmates/…
- **One** 8.5" x 11" (double-sided) cheat sheet is allowed
- You may bring a calculator (optional & recommended)
- Your **better** exam counts for **2/3** of your exam grade.

Scope

- Approximate Inference
  - Lecture 7.2, Classwork 5
- (Hidden) Markov Models
  - Lecture 8.2 – 9.2, Classwork 5
- Machine Learning & Naïve Bayes Classifier
  - Lecture 10.1 – 11.1, Classwork 6
- Linear Perceptron, Logistic Regression, & Neural Networks
  - Lecture 11.2 – 14.1, Classwork 6

Final Exam Style

- 7 Questions
  - 6 classwork-style questions
  - 1 question contributed by the class
- “Disclaimers”
  - Final Exam
  - (Optional) Final Exam Practice
    - May look different from the actual exam
    - Applicable to our section only

slido

Please download and install the
Slido app on all computers you use



How would you like the questions to be formatted?

ⓘ Start presenting to display the poll results on this slide.

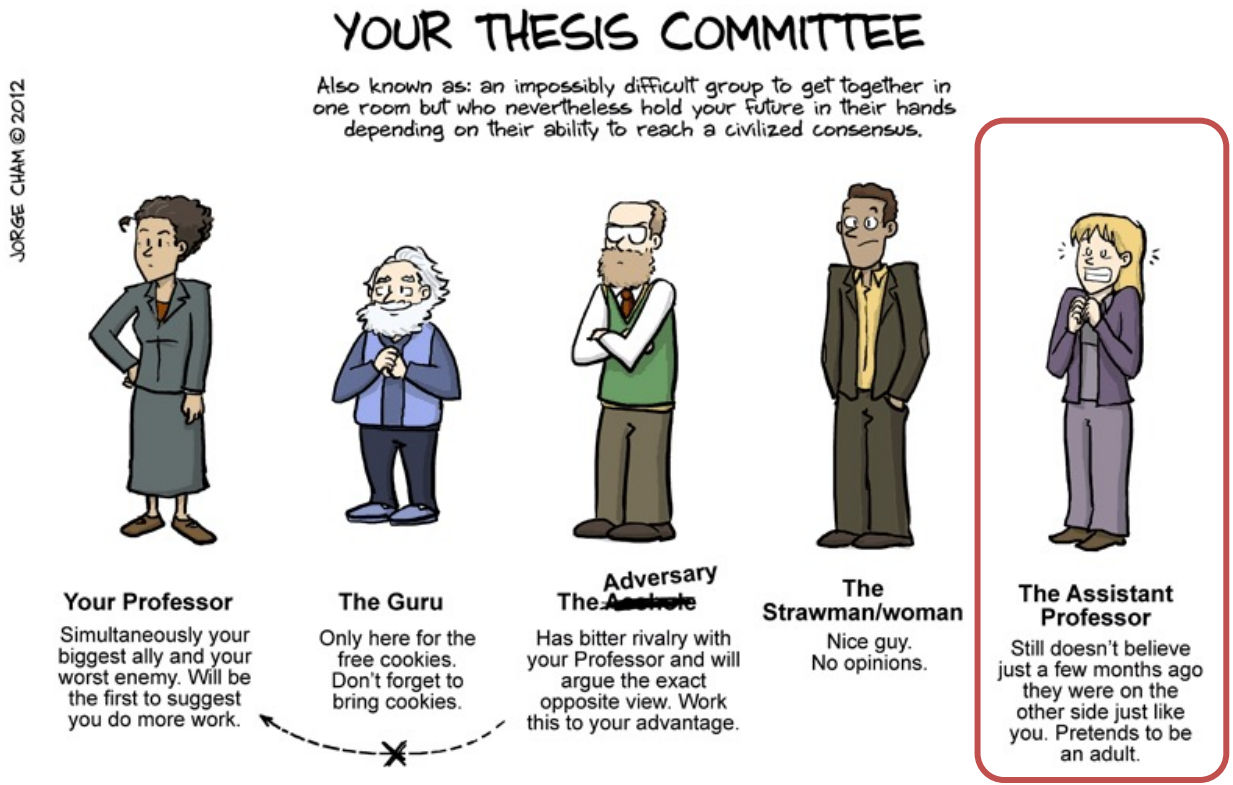
Review Material / Resource

- Lecture Slides
- Classworks
- Final Exam Practice
- Office Hours During Final Week
  - Tue/Thu 10 am - Noon

Notes

- Maintaining the Keck Lab
  - Please keep the lab space clean and tidy
- Research and TA Opportunities
  - Please feel free to reach out to me if you are interested in
    - AI-related research in the Spring/Summer
    - TA opportunities for CMSI 3300 in the Spring

Course Evaluation (Email/Brightspace)

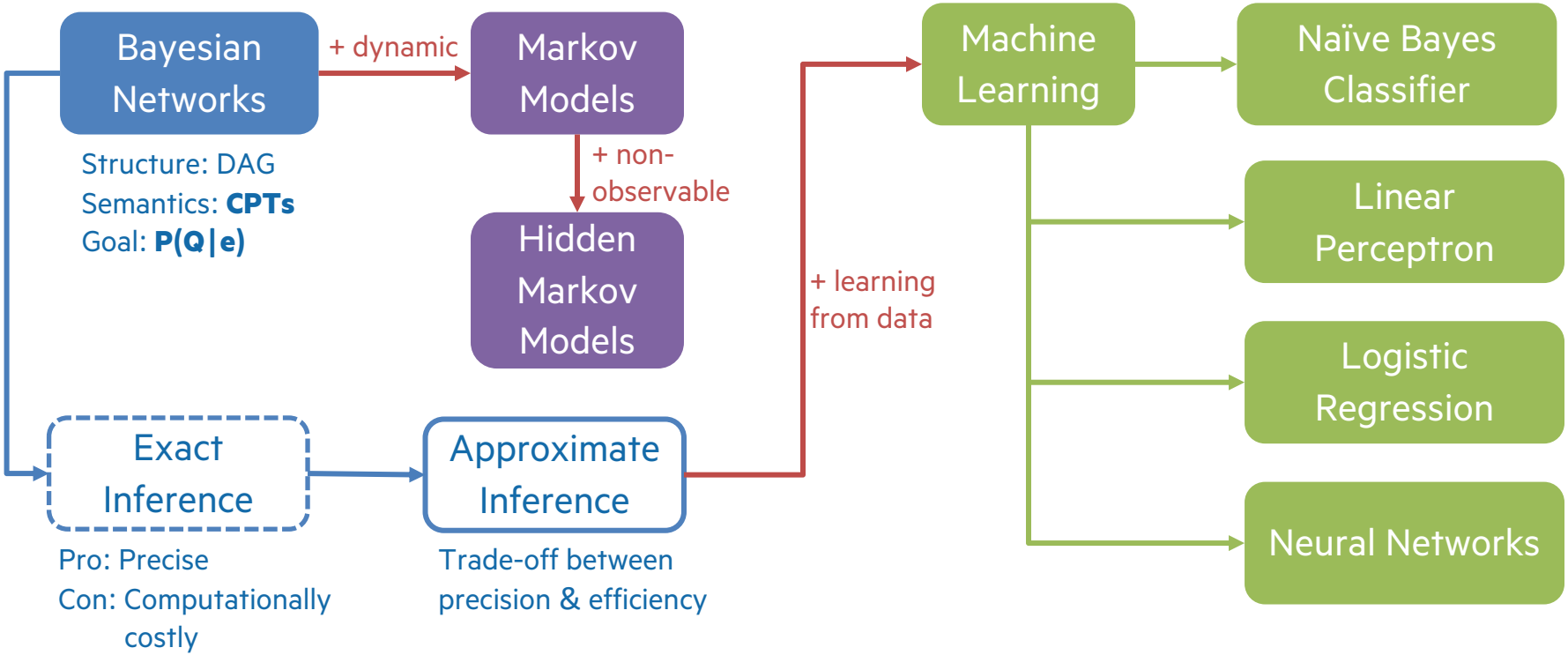




Next …

FINAL EXAM REVIEW

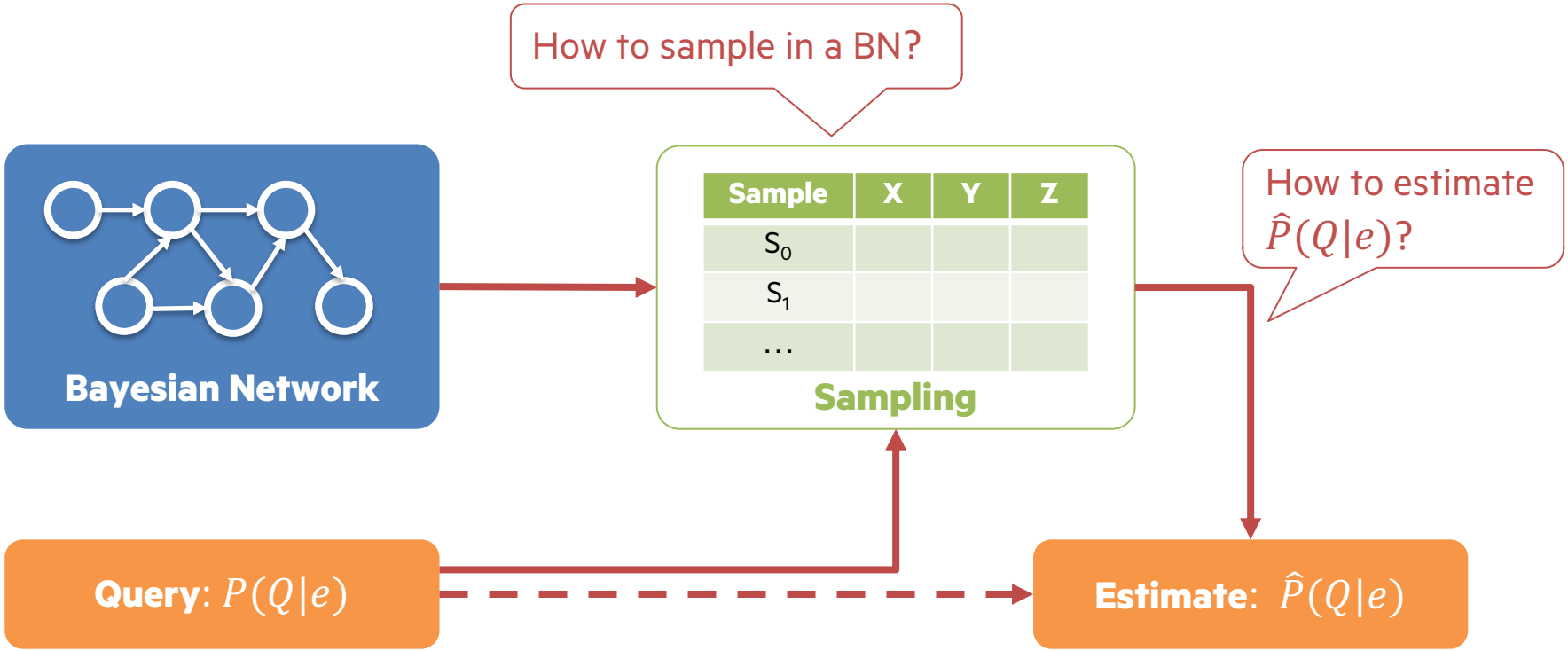
Overview


Bayesian Networks
+ dynamic
Markov Models
+ non-observable
Hidden Markov Models
Structure: DAG
Semantics: CPTs
Goal: P(Q|e)
Exact Inference
Approximate Inference
Pro: Precise
Con: Computationally costly
Trade-off between precision & efficiency
+ learning from data
Machine Learning
Naïve Bayes Classifier
Linear Perceptron
Logistic Regression
Neural Networks

1. Approximate Inference

- ❑ Sampling in BN
- ❑ Prior Sampling
  - ❑ Topological Sort
  - ❑ Rejection Sampling
- ❑ Gibbs Sampling
  - ❑ Steps

Sampling in a BN



Prior Sampling & Rejection Sampling

- Given the BN structure and semantics
  - CPT: P(v | parents(v))
    - Sample **parents** first, then **child**
  - Proceduralize via **Topological Sort:**
    - An ordering of nodes in which a **node never appears before its parents** already have in the sequence.
- Rejection Sampling
  - Any sample **inconsistent** with evidence is immediately ignored / thrown out.

Gibbs Sampling

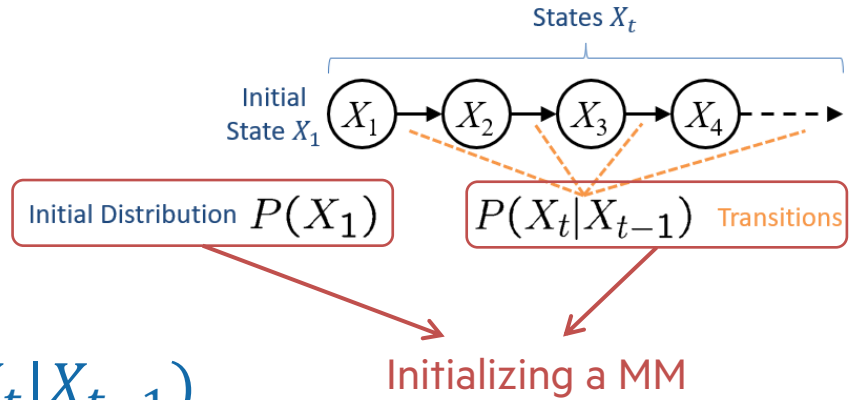
- Fix the evidence from the start
  - Sampling from some variable V given *all other* variables in the BN is much easier than sampling from the JPT itself.
- Steps
  - Initialize all variables to some random assignment, except for the evidence e, which remains fixed to its value.
  - Sample a single variable at a time conditioned on the rest (again keeping e fixed).
  - Repeat Step 2 many times.

2. (Hidden) Markov Models

- ❑ Markov Models
  - ❑ Initialization
  - ❑ Joint Distribution Factorization
  - ❑ Mini-Forward Algorithm
- ❑ Hidden Markov Models
  - ❑ Initialization
  - ❑ Forward Algorithm (Filtering)
  - ❑ Particle Filtering

Markov Models

- Markov Models
  - **State** Variable: X_t
    - Initial Distribution: $P(X_1)$
  - **Transition** Probabilities: $P(X_t|X_{t-1})$
  - What's the **goal** of MM?
    - Predicting **future** states <u>only</u> on the **current** state
  - **Markov property**
    - **future** $\perp\!\!\!\perp$ **past** $|$present
    - It is a chain and d-separation applies



Markov Models

- Joint Distribution Factorization

$$P(X_1, X_2, \dots, X_T) = P(X_1) \prod_{t=2}^{T} P(X_t | X_{t-1})$$

- Mini-Forward Algorithm (for Arbitrary $P(X_t)$)

$$P(X_t = x_t) = \sum_{x_{t-1} \in X_{t-1}} P(X_t = x_t | X_{t-1} = x_{t-1}) P(X_{t-1} = x_{t-1})$$

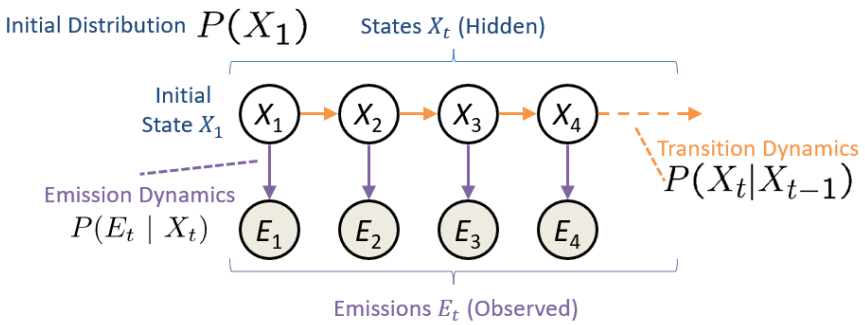
All possibilities of previous state

transition probability

state likelihood at **t-1**

Hidden Markov Model

- HMM
  - Components
    - State X and Emission E
    - Emission Dynamics
    - Transition Dynamics
  - Assumptions
    - States are **not observed**
    - Emissions are **observed**



Hidden Markov Model

- Inference Goal of HMM
  - $B(X_t) = P(X_t|e_{1:t})$
  - $B(X_t)$: Belief State
- Forward Algorithm (Filtering)
  - $B(X_t = x_t) = P(e_t|x_t) \sum_{x_{t-1}} P(x_t|x_{t-1}) \cdot \alpha B(x_{t-1})$

emission dynamics

transition dynamics

belief of being in last state

Forward Algorithm as Dynamic Programming

- Computing $B(X_t)$ follows **repeated** application of **two steps** at all previous time steps:
  - Time Update

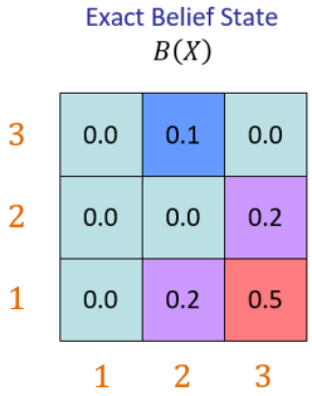
$$B'(x_t) = P(x_t|e_{1:t-1}) = \sum_{x_{t-1}} P(x_t|x_{t-1}) \cdot B(x_{t-1})$$

  - Observation Update

$$B(x_t) = P(x_t|e_{1:t}) \propto P(e_t|x_t)B'(x_t)$$

Particle Filtering

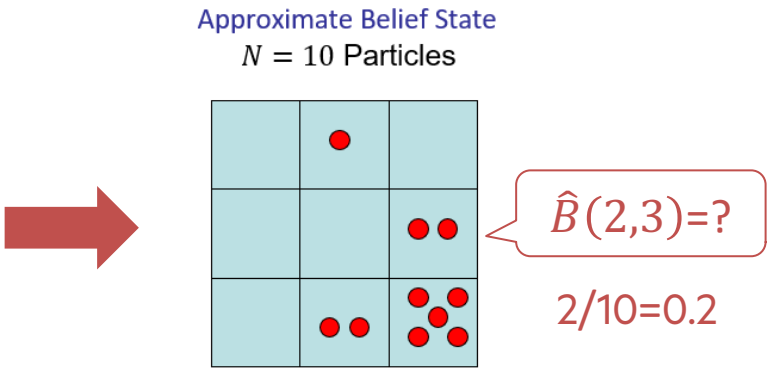
- Representing the belief state $B(X)$ as **a list of N particles**, which are samples of $B(X)$ that aggregate around the most likely states.



State $X = (row, col)$:
location of ghost

Transition $P(X_t|X_{t-1})$:
movement of ghost

Emission $P(E_t|X_t)$:
sensor reading



Particle Filtering

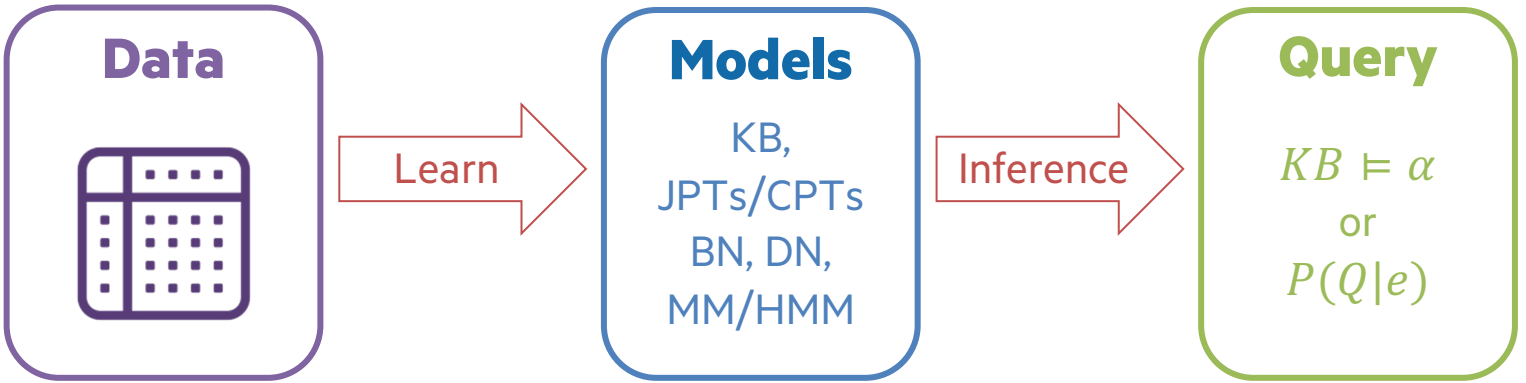
- Step 1 – Elapse Time
  - $x_{t+1} = sample(P(X_{t+1}|X_t))$
- Step 2 – Observing Evidence e
  - Downweight: $B(X) = P(e_t|x_t) * B'(X)$
- Step 3 – Resampling/Normalization
  - Resample with replacement
- Step 4: Repeat

3. Machine Learning Basics

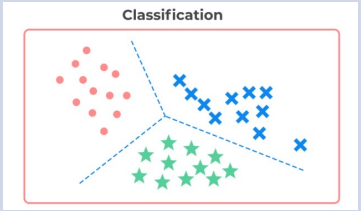
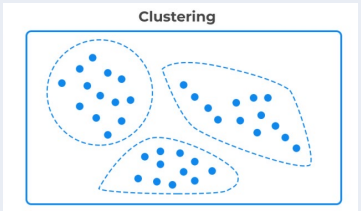
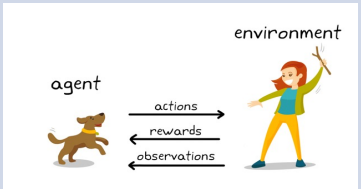
- ❑ Types of Learning
  - ❑ Supervised
  - ❑ Unsupervised
  - ❑ Reinforcement
- ❑ Basics
  - ❑ Regression vs. Classification
  - ❑ Training, Validation, and Test
  - ❑ Overfitting vs. Underfitting
  - ❑ Parameter vs. Hyperparameter
  - ❑ Learning vs. Tuning
  - ❑ Evaluation Metrics

Machine Learning

- A bottom-up approach
  - The process of computationally discovering/constructing a model from data / experience.
- Learning vs. Inference

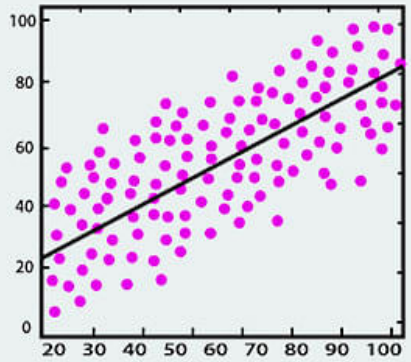


Types of Learning

Type	Feedback	Data	Goal	Example
Supervised Learning	Immediate & explicit	Labeled (input-output)	Find mapping between inputs and outputs	
Unsupervised Learning	No feedback	Unlabeled	Find structure / patterns in data	
Reinforcement Learning	Delayed & reward-based	No predefined data, but need rewards	Maximize rewards	

(Supervised) Regression vs. Classification

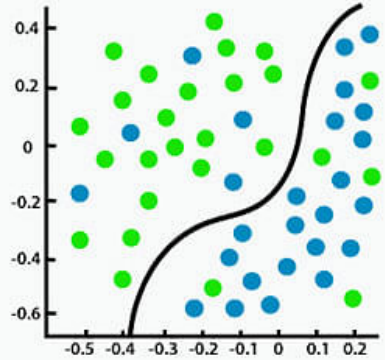
Output:
continuous numerical values



Regression

versus

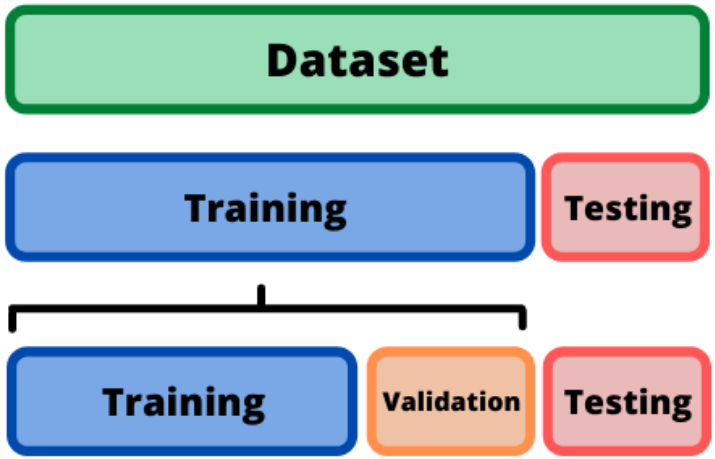
Output:
discrete categories/labels



Classification

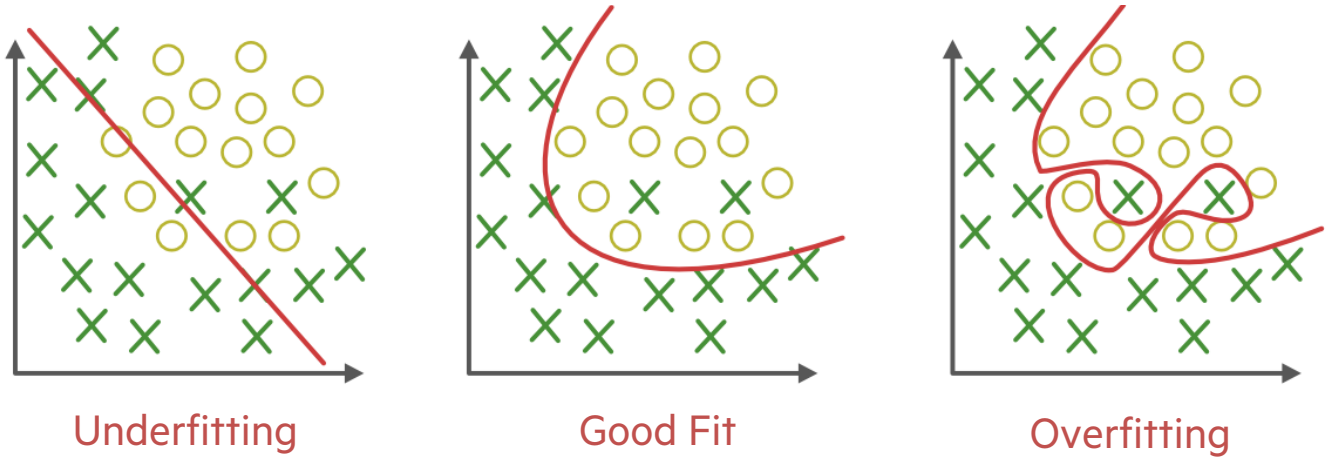
Training, Validation, and Test

- Training (~70%)
  - Train/learn the model
- Validation (~20%)
  - Tune hyperparameters
  - Evaluate during training
- Test (~10%)
  - For final model evaluation
  - Unbiased estimate of model performance



Overfitting vs. Underfitting

- Classification

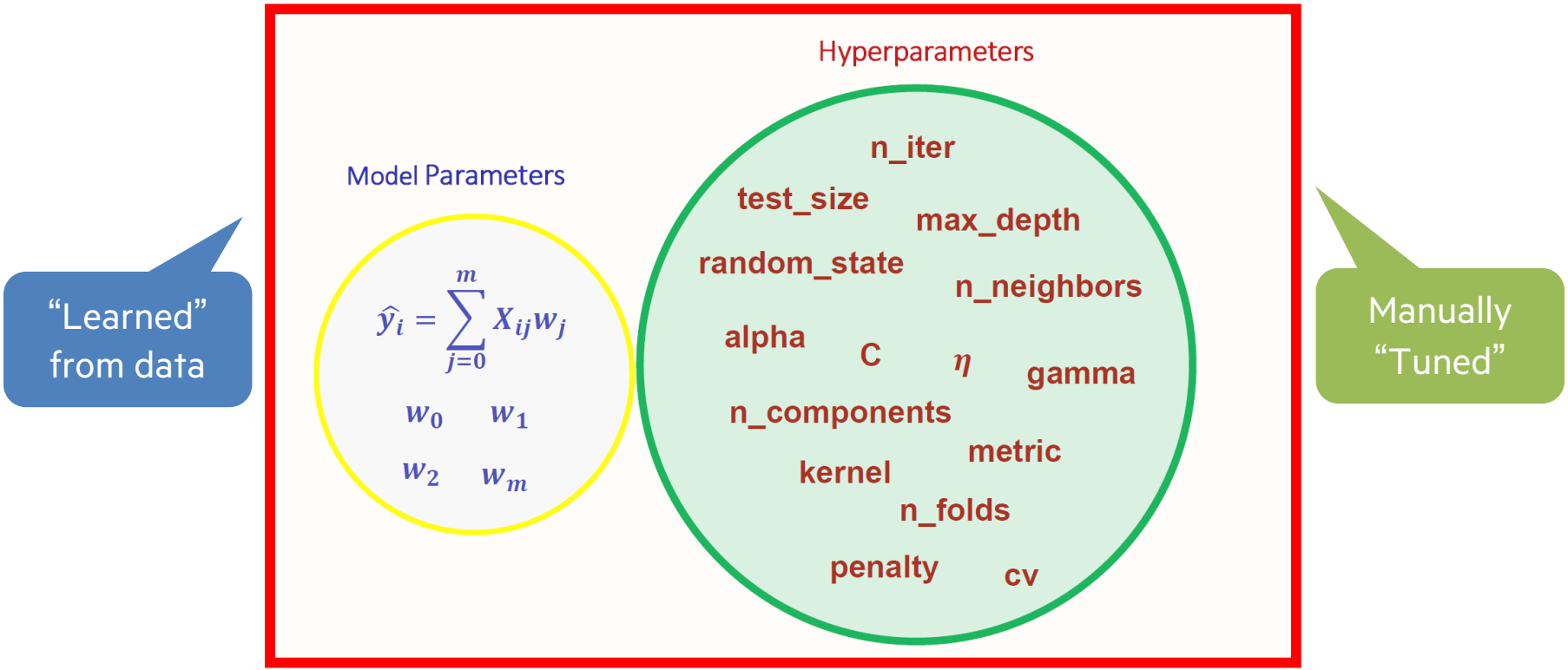


Overfitting vs. Underfitting

- Regression



Parameter vs. Hyperparameter



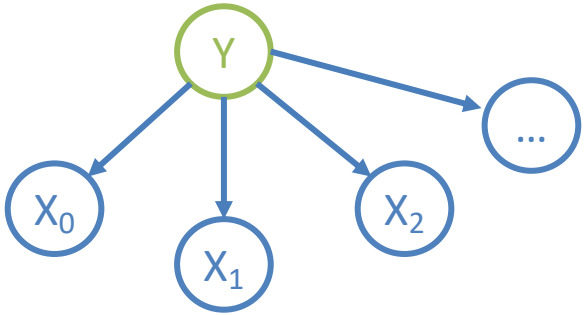
Evaluation

- Metrics
  - Classification
    - Confusion Matrix
    - Accuracy
    - Precision
    - Recall
    - F1 Score
  - Regression
    - MSE
    - RMSE
    - MAE
    - R^2

4. Naïve Bayes Classifier

- ❑ Structure & Semantics
- ❑ Inference Goal
- ❑ Decision Rule
- ❑ Overfitting
- ❑ Smoothing
- ❑ Hyperparameter Tuning

Naïve Bayes Classifier



- Naïve Bayes Classifier (NBC)
  - Structure
    - Class Y is the parent of all feature nodes $X_i \in X$
  - Semantics
    - $P(X_i|Y) \forall X_i \in X$
  - Goal
    - Find the most likely y from X
    - $\hat{y} = argmax_{y \in Y} P(Y = y | X_0 = x_0, X_1 = x_1, \dots)$

Naïve Bayes Classifier

- Decision Rule

$$\hat{y} = argmax_{y \in Y}\, P(Y = y | X_0 = x_0, X_1 = x_1, \dots)$$
$$= argmax_{y \in Y} P(y) \prod_{x_i} P(x_i | y)$$
$$= argmax_{y \in Y} \log\big(P(y)\big) + \sum_{x_i} P(x_i | y)$$

- "Learning" from data
  - $P(x_i|y) = \frac{c(x_i)}{N} = \frac{\#\text{ occurance of } x_i}{\#\, samples}$

Aka Maximum Likelihood Estimation (MLE)

Overfitting in NBC

- NBC for spam detection
  - What may cause overfitting in NBC?
  - Limited Training Data
    - MLE estimation $P(x_i|y) = \frac{c(x_i)}{N}$ is less accurate
  - Zero Probability Problem: $P(x_i|y) = 0$ for word x_i in the test set that never appears in the training set
    - $\hat{y} = argmax_{y \in Y} P(y) \prod_{x_i} P(x_i|y) = 0 \; \forall y \in Y$

How to fix the Zero Probability Problem?

Smoothing

- Produce more **generalizable models** in machine learning by artificially inserting some information.
- Smoothing in NBC

$$P(x_i|Y = spam) = \frac{N_{x_i|spam} + \alpha}{N_{spam} + \alpha \cdot N_{vocab}}$$

$$P(x_i|Y = ham) = \frac{N_{x_i|ham} + \alpha}{N_{ham} + \alpha \cdot N_{vocab}}$$

α is the **smoothing weight** to prevent zero probability problem/overfitting

How to determine the value of α?

Hyperparameter Tuning

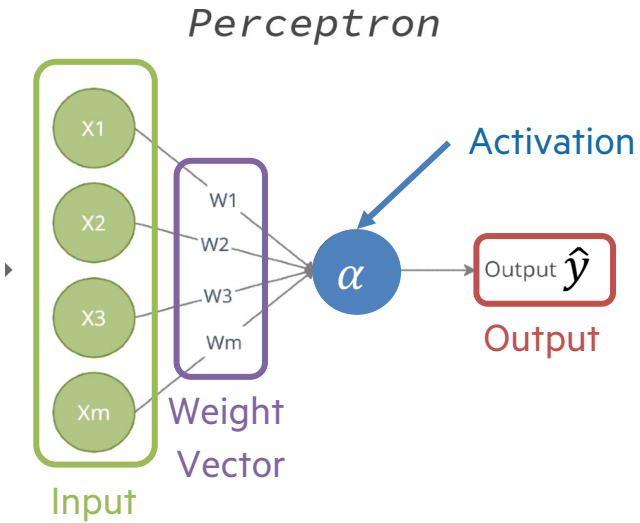
- Find the optimal values for a model's hyperparameters.
- **Parameters** and **Hyperparameters** in NBC
  - **Parameters**: automatically learned from the training data
    - CPTs: $P(y), P(x_i|y)$
  - **Hyperparameters** : manually tuned via searching/optimizing
    - Smoothing weight: α

5. Linear Perceptron

- ❑ Parameters
- ❑ Linear Perceptron Update Algorithm
- ❑ Perceptron Properties
  - ❑ Separability
- ❑ Problems
  - ❑ Convergence
  - ❑ Generalization
  - ❑ Overfitting

Perceptron

- Weight Vector $\boldsymbol{w}$
  - $\boldsymbol{w} =< w_1, w_2, \dots, w_n >$
- Activation α
  - $\alpha = \boldsymbol{w} \cdot \boldsymbol{x} = \sum_i w_i \cdot x_i$
- Decision Rule
  - Binary: $\hat{y} = (\text{activation}_{\boldsymbol{w}}(\boldsymbol{x}) > 0)?\, 0 : 1$
  - Multiclass: $\hat{y} = \text{argmax}_y \alpha_y = \text{argmax}\, \boldsymbol{x} \cdot \boldsymbol{w_y}$



Where do they come from?

Linear Perceptron Update Algorithm

Initialize Weights

```
# Initialize all class' weight vectors to all 0s
w_y = <0, 0, ..., 0>
```

Predict w/ Current Weights

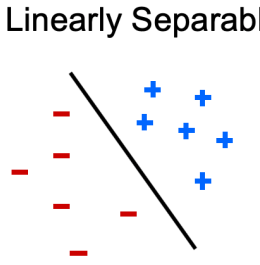
```
# Go through each example in training set
for each x in training_set:
    # Make prediction with current weights
    y_hat = argmax_y (w_y • x)
```

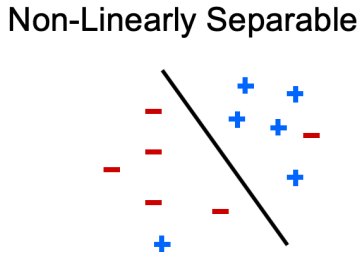
Update Weights

```
    # Let's call y* the "right" answer from training
    # If our prediction y_hat is right, yay! Do nothing... else:
    if y_hat != y*:
        # Reduce weights of false positive
        w_{y_hat} = w_{y_hat} - x
        # Increase weights of correct answer
        w_y* = w_y* + x
```

Perceptron Properties

- Separability
  - Linearly Separable
    - **Guarantee convergence**
    - Does not guarantee the most **generalizable** decision boundary
  - Non-linearly Separable
    - **Prevent Convergence**





6. Logistic Regression

- ❑ Parameters
- ❑ Objective & Loss
- ❑ Optimization & Learning
  - ❑ Gradient Descent
  - ❑ Stochastic Gradient Descent

Linear Perceptron vs. Logistic Regression

	Linear Perceptron	Logistic Regression
Input	$\boldsymbol{x} =< x_1, x_2, \cdots, x_n >$	$\boldsymbol{x} =< x_1, x_2, \cdots, x_n >$
Weight Vector	$\boldsymbol{w} =< w_1, w_2, \cdots, w_n >$	$\boldsymbol{w} =< w_1, w_2, \cdots, w_n >$
Activation	$\alpha = \boldsymbol{w} \cdot \boldsymbol{x} = \sum_i w_i \cdot x_i$	Binary: $\alpha = \sigma(z) = \frac{1}{1+e^{-z}}$, $z = \boldsymbol{w} \cdot \boldsymbol{x}$ Multi-Class: $\frac{e^{z_y}}{\sum_{y'} e^{z_{y'}}}$, $z_y = \boldsymbol{w_y} \cdot \boldsymbol{x}$
Decision Rule	Binary: $\hat{y} = (\alpha_{\boldsymbol{w}}(\boldsymbol{x}) > 0)?\, 0 : 1$ Multi-Class: $\hat{y} = \text{argmax}_y \alpha_y = \text{argmax}\, \boldsymbol{x} \cdot \boldsymbol{w_y}$	Binary: $\widehat{y^{(i)}} = \left(P\left(y^{(i)} = +1 \mid \boldsymbol{x}^{(i)}; \boldsymbol{w}\right) > 0.5\right)? +1 : -1$ Multi-Class: $\widehat{y^{(i)}} = \text{argmax}_{y^{(i)}} P\left(y^{(i)} \mid \boldsymbol{x}^{(i)}; \boldsymbol{w_y}\right)$
Optimization	For each misclassification: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}$	Gradient Descent $w_{k,i} = w_{k,i} - \eta \cdot \frac{\partial L_{CE}}{\partial w_{k,i}}$

Objective and Loss

- From training data: pairs of $(\boldsymbol{x}^{(i)}, y^{(i)})$
- Loss function
  - $Loss(\text{predicted}, \text{true}) = Loss(\hat{y}^{(i)}, y^{(i)})$
  - It provides a numerical metric of how wrong the model is.
- Cross Entropy (CE) Loss
    - $Loss_{CE}(\hat{y}^{(i)}, y^{(i)}) = -\log P(\hat{y}^{(i)}|\boldsymbol{x}^{(i)}; \boldsymbol{w})$
    - Commonly used for classification

Gradient Descent

- Weight Update
  - Goal: maximize $ll(\boldsymbol{w})$ => **minimize** loss function
  - The derivative of a **loss function** w.r.t. **weights** indicates the direction to *nudge* those weights to be better!
- Derivative of $Loss_{CE}$ (L_{CE}) w.r.t. a class k and a sample i

$$\frac{\partial L_{CE}}{\partial w_{k,i}} = -\left[1\left(k = y^{(i)}\right) - P\left(\hat{y}^{(i)} = k \middle| \boldsymbol{x}^{(i)}; \boldsymbol{w_k}\right)\right] \cdot \boldsymbol{x}^{(i)}$$

Direction to move $w_{k,i}$ to

Indicator function =1 if $k = y^{(i)}$, =0 otherwise

Tweak proportional to the contribution of each feature

Gradient Descent

- Logistic Regression Weight Update Rule:
  - For a single sample $(x^{(i)}, y^{(i)})$, weights belonging to each class w_y can be nudged in the "more correct" direction
  - $w_{k,i} = w_{k,i} - \eta \cdot \frac{\partial L_{CE}}{\partial w_{k,i}}$

Current weight

Learning Rate

Direction

  - Demo: https://uclaacm.github.io/gradient-descent-visualiser/

Stochastic Gradient Descent (SGD)

- Steps

```
initialize w_y = <0, 0, 0, ...>

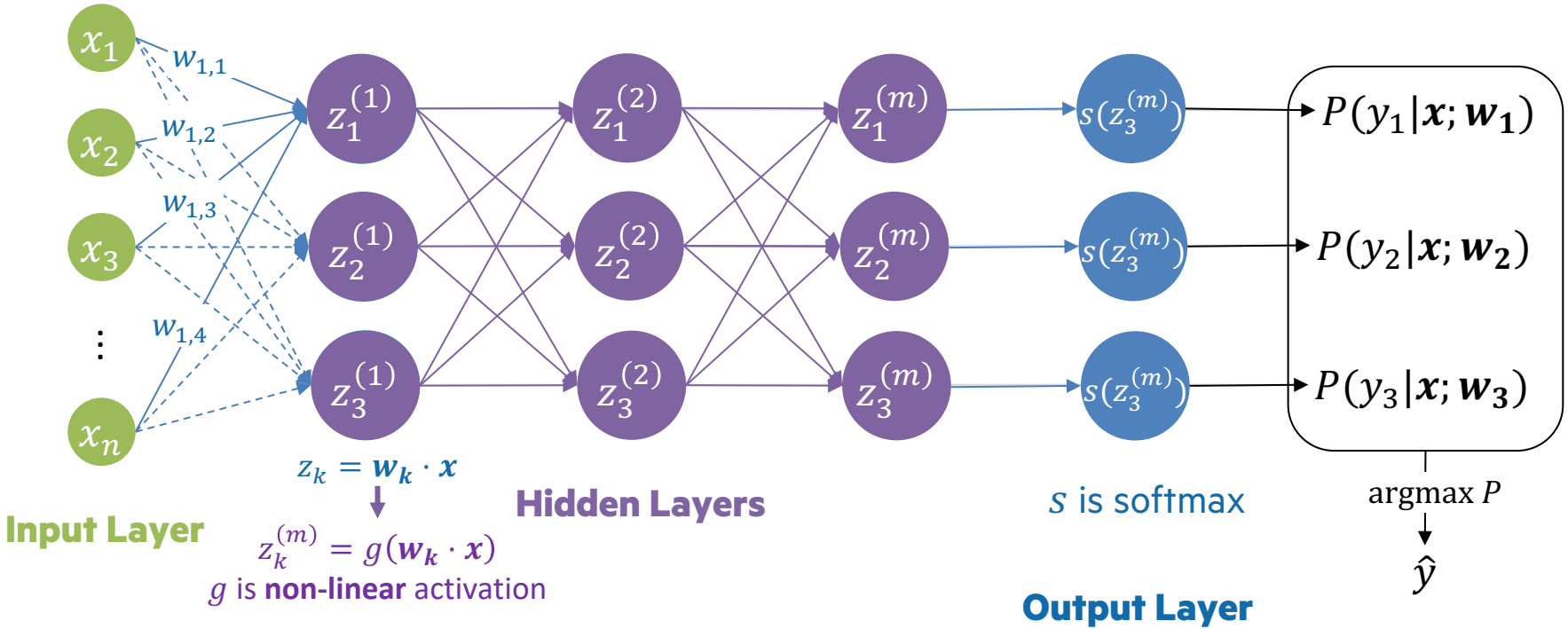
# avoid over training
while validation set accuracy still increasing:
  for each sample (X^(i), y^(i)) in training set (in random order):
    # compute the current prediction
    compute P(Y|x^(i); w)
    # gradient
    compute gradient (derivatives) of Loss_CE for each w_{k,i}
    # update
    update weight of each class: w_{k,i} = w_{k,i} - learning_rate * gradient
```

7. Neural Networks

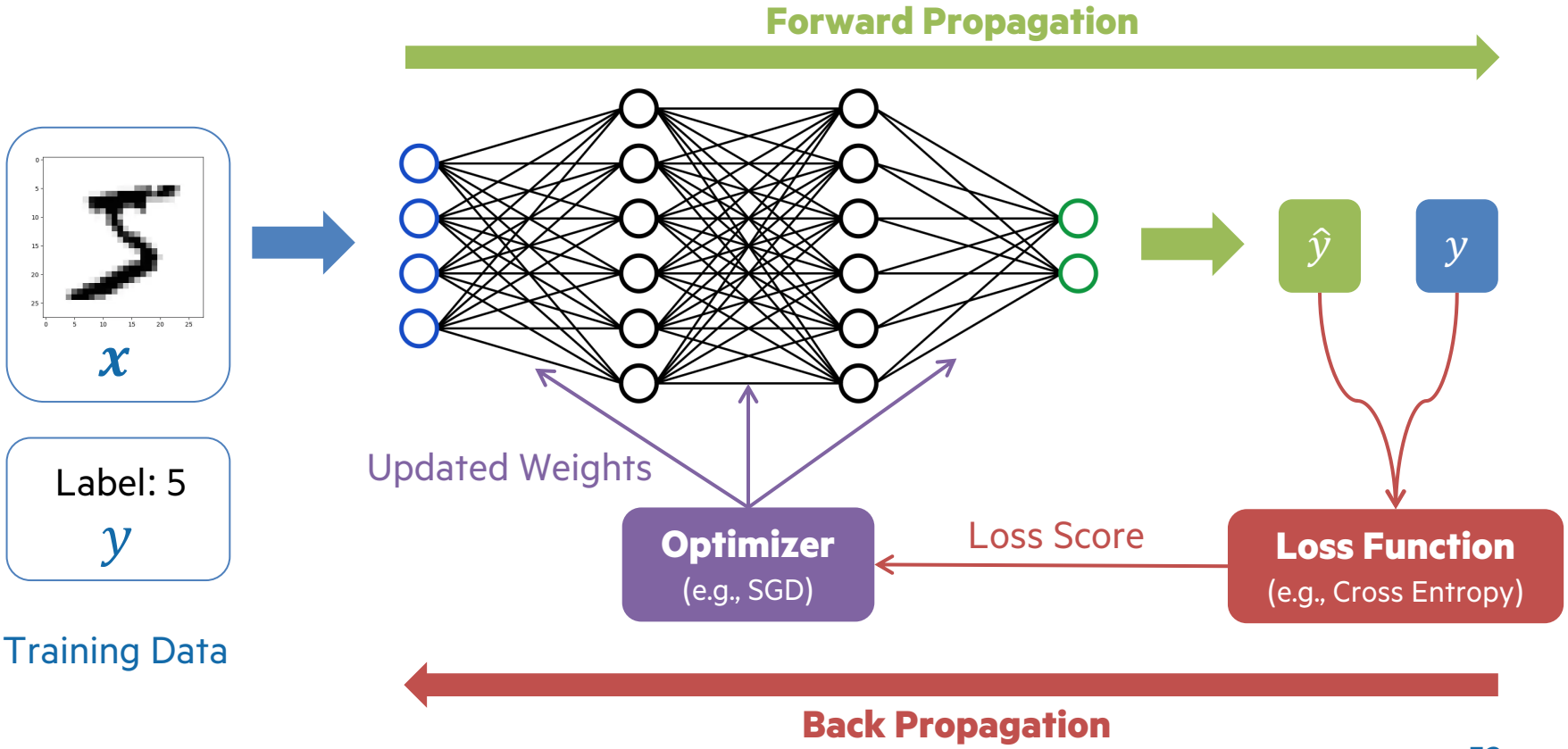
- ❑ Feed-Forward Network
  - ❑ Layers
    - ❑ Input Layer
    - ❑ Hidden Layers
    - ❑ Output Layer
  - ❑ Non-linear activation
  - ❑ Loss & Back Propagation

Neural Networks

- Feed-Ford Network



Learning NN


Forward Propagation
x
Label: 5
y
Training Data
$\hat{y}$
y
Updated Weights
Optimizer
(e.g., SGD)
Loss Score
Loss Function
(e.g., Cross Entropy)
Back Propagation

Good Luck!


THE STUDENT VIEW:
AHH! I HAVE TO TAKE AN EXAM TOMORROW!
JORGE CHAM © 2018
THE PROFESSOR VIEW:
AHH! I HAVE TO GIVE AN EXAM TOMORROW!
WWW.PHDCOMICS.COM